

A Security Assessment of iSpy Open-Source Video Surveillance Software

Simon Richards

Sreeja Badri

Aryan Kumar

Dinesh Melimi

Abstract—This paper presents a partial security assessment of iSpy, an open-source video surveillance application built on C# and the .NET Framework. The assessment targets the application’s local credential store, the shared-secret authentication mechanism enforced by the embedded HTTP server, and the absence of brute-force protections. Source-level analysis of the project’s GitHub repository, supplemented by dynamic interaction with a locally built instance, identified four high-impact issues: (1) user passwords are stored using a reversible AES round-trip rather than a one-way hash; (2) the encryption used for that round-trip and for related sensitive configuration fields relies on hardcoded keys and a hardcoded salt that are visible in the public source code; (3) the embedded web server authenticates clients with a single shared GUID transmitted in URL query parameters; and (4) no rate limiting, lockout, or attempt logging is applied to failed authentications. Each finding is mapped to a CWE category. Working patches that address the credential storage flaw and the absence of rate limiting are implemented and documented, and the cryptographic behaviour of the new password hasher is validated empirically. The results suggest that iSpy’s local-trust authentication model is fragile in any deployment where the configuration file or the iSpy host is not fully trusted.

Index Terms—security assessment, iSpy, password storage, PBKDF2, hardcoded keys, authentication, rate limiting, CWE, code review, surveillance software

I. INTRODUCTION

iSpy is a long-running open-source video surveillance application that records and streams audio and video from webcams and IP cameras, with optional remote access via an embedded HTTP server. The full source is hosted at <https://github.com/ispysoftware/iSpy>, and prebuilt installers are distributed from <https://www.ispyconnect.com/>. The Windows installer packages the iSpyApplication binary and its native dependencies; the source repository contains roughly 1,200 files, including approximately 400 C# source files spanning the desktop UI, device drivers, the embedded server, and supporting utilities.

Because iSpy handles continuous audiovisual recordings, exposes a network-accessible control surface, and is frequently deployed by non-specialist users in residential and small-business environments, weaknesses in its credential handling translate directly into privacy risk for end users. No published peer-reviewed audit of iSpy’s authentication subsystem was identified during preparation of this report, motivating a partial assessment scoped to that subsystem.

The remainder of this paper is structured as follows. Section II describes the components of iSpy that are relevant to authentication. Section III presents the threat model. Section IV reviews the security architecture of the credential

store, the desktop login flow, and the embedded HTTP server’s auth mechanism. Section V documents the vulnerabilities discovered through static and dynamic analysis and describes the two prototype fixes implemented. Section VI summarises the findings and the lessons drawn from the exercise.

II. DESCRIPTION OF THE SOFTWARE

A. Overall architecture

iSpy is a Windows desktop application targeting .NET Framework 4.8, with a WinForms UI in `MainForm.cs` and an embedded HTTP server in `Server/LocalServer.cs`. Configuration is stored as XML on the local filesystem and serialised through the autogenerated classes in `XML/config.cs`.

B. Camera and recording engine

The application supports DirectShow webcams, ONVIF/RTSP/MJPEG IP cameras, and microphones. Per-device configuration, motion detection parameters, alerts, and recording paths are persisted in the same XML configuration file as user accounts.

C. User accounts and the credential store

Users (referred to internally as “permission groups”) are defined by the `configurationGroup` type in `XML/config.cs`. Each record carries a name, a feature-set integer, and a password field that holds the credential in the form persisted to disk. The desktop lock screen is implemented in `CheckPassword.cs` and is the primary user-facing authentication surface.

D. Embedded HTTP server

`Server/LocalServer.cs` (4,259 lines) implements a custom HTTP listener used for both the local web UI in `WebServerRoot/` and for authenticated callbacks from the iSpy web service. After `MainForm` starts, it generates a session-local GUID (`MainForm.Identifier`, see Listing 1) which is treated as a shared secret: any HTTP request whose `auth` query parameter equals this GUID is considered authenticated.

```
Identifier = Guid.NewGuid().ToString();
```

Listing 1. Per-process auth token (`MainForm.cs` line 1023)

E. Cryptography utility

The class `EncDec` in `EncDec.cs` provides `EncryptData` and `DecryptData` helpers built on `RijndaelManaged` with a key derived via `PasswordDeriveBytes`. This single utility is used for several distinct purposes throughout the code base, including the password field of every user account.

III. THREAT MODEL

A. Assets

The protected assets in an iSpy deployment are: (a) the live and archived audiovisual streams; (b) user account credentials persisted in the XML configuration; (c) authentication tokens¹ for the upstream ispyconnect web service (`WSPassword`);² and (d) the iSpy host operating system, which a privileged attacker could pivot from after gaining administrative access to the application.

B. Threat actors

We consider three actor profiles. *T1 – Local file-system adversary*: a process or user able to read the iSpy `AppData` directory, e.g. through a malware foothold or a backup leak. *T2 – Network adversary on the LAN*: an attacker on the same network segment who can observe HTTP traffic to the embedded server. *T3 – Remote unauthenticated attacker*: an attacker who has discovered a port-forwarded iSpy instance on the public internet (a common deployment pattern).

C. STRIDE summary

TABLE I
STRIDE THREATS MAPPED TO ISPY AUTHENTICATION

Category	Applicable threat
Spoofing	Reuse of the leaked shared GUID to impersonate any user (T2, T3).
Tampering	Direct edit of the XML configuration to install or change accounts (T1).
Repudiation	Authentication events are logged with username only; no IP, no failure log.
Info. Disclosure	Plaintext recovery of every stored password from the config file plus public source (T1).
DoS	Unlimited failed-auth processing on the HTTP listener (T2, T3).
Elev. of Privilege	Single-token auth grants full admin-equivalent access on success.

IV. SECURITY ARCHITECTURE AND DESIGN REVIEW

A. Credential storage design

The desktop login path in `CheckPassword.cs` validates the password as shown in Listing 2.

```

1 if (txtPassword.Text ==
2   EncDec.DecryptData(g.password, MainForm.Conf.EncryptCode
3   ))
4   // login successful
5 }
```

Listing 2. Login comparison (`CheckPassword.cs` line 19)

The stored `g.password` is therefore an AES-encrypted ciphertext that is decrypted to plaintext on every login attempt and compared by string equality. This design choice is the central architectural problem of the credential store: any party able to read the configuration file and the encryption key recovers every password as plaintext, with no cracking required.

B. Key material used for the credential store

The key passed to `EncDec` for user accounts is `MainForm.Conf.EncryptCode`, a per-installation GUID generated on first run. However, `EncDec` itself derives its AES key with a hardcoded literal salt, shown in Listing 3.

```

var pdb = new PasswordDeriveBytes(
    password, Encoding.UTF8.GetBytes("Salt"));
```

Listing 3. Hardcoded salt (`EncDec.cs` lines 73 and 104)

The literal string `"Salt"` is identical across every iSpy installation worldwide, eliminating the protection a salt is meant to provide. In addition, several sensitive fields outside the user account list – including `WSPassword` (the upstream web-service credential) and `ChosenGroupName` – are encrypted with a *hardcoded GUID literal* embedded directly in the source, shown in Listing 4.

```

_conf.WSPassword = EncDec.DecryptData(_conf.WSPassword,
    "582df37b-b7cc-43f7-a442-30a2b188a888");
```

Listing 4. Hardcoded master key (`MainForm_Configuration.cs` line 365)

Because the hardcoded GUID is in the public repository, the confidentiality of `WSPassword` on disk is reduced to the secrecy of the AES algorithm, i.e. none.

C. Embedded server authentication design

`LocalServer.CheckAuth` (Listing 5) is the single gatekeeper for HTTP-served content. It performs a string comparison between the request's `auth` query parameter and the process-wide `MainForm.Identifier` GUID.

```

1 private static bool CheckAuth(string sPhysicalFilePath)
2 {
3     return GetVar(sPhysicalFilePath, "auth")
4     == MainForm.Identifier;
5 }
```

Listing 5. Server auth check (`LocalServer.cs` line 3250)

This design has three consequences. First, the same shared secret authorises every authenticated request, so there is no notion of per-user identity over HTTP. Second, the secret is a `Guid.NewGuid()` value, which is documented as not cryptographically random; it provides roughly 122 bits of effective entropy but is not generated by a CSPRNG. Third, callers in `Controls/CameraWindow.cs` (line 3284) and `Controls/VolumeLevel.cs` (line 1633) demonstrably append the token to URLs (`...&auth=<GUID>`), so the secret enters server access logs, browser history, and any HTTP `Referer` headers generated when a user navigates away from the page.

D. Absence of rate limiting

A search of the codebase for the strings `lockout`, `attempts`, `TooMany`, `429`, and related tokens returned no matches anywhere in the application logic. The login path in `CheckPassword.cs` processes attempts synchronously and the HTTP `CheckAuth` path performs no per-IP accounting whatsoever.

V. CODE ANALYSIS AND PENETRATION TESTING

A. Methodology

The repository was cloned at its current HEAD on the public GitHub mirror. Static analysis was conducted by direct source review and targeted `grep` sweeps for cryptographic primitives (MD5, SHA1, Rfc2898, `bcrypt`, `Argon2`), credential identifiers, and known-bad patterns (hardcoded keys and salts). Dynamic interaction was performed against a locally built instance to confirm token formats and request flows.

B. Findings

F-01 – Reversible AES used for password storage (Critical). *CWE-257: Storing Passwords in a Recoverable Format.* The credential field of every user account is stored as the output of `EncDec.EncryptData` and decrypted on every login (`CheckPassword.cs:19`). Anyone with read access to the iSpy AppData directory and the `EncryptCode` GUID – both of which sit in the same file – recovers every plaintext password instantly. This is materially worse than a fast-hash flaw such as unsalted MD5: there is no cracking step at all.

F-02 – Hardcoded literal salt and master key (Critical). *CWE-760: Use of a Predictable Salt; CWE-798: Use of Hard-coded Credentials.* `EncDec.cs` derives every AES key it produces from the literal four-byte salt "Salt", and `MainForm_Configuration.cs` encrypts `WSPassword` and related fields with the hardcoded GUID `582df37b-b7cc-43f7-a442-30a2b188a888` pasted directly into the source. Both values are public.

F-03 – Auth token transmitted in URL query string (Medium). *CWE-598: Information Exposure Through Query Strings in GET Request.* `Controls/CameraWindow.cs`⁹ and `Controls/VolumeLevel.cs` both build URLs¹ of the form `...&auth=<MainForm.Identifier>`. The same token is the sole credential checked by `LocalServer.CheckAuth`; once it appears in any access log, browser history entry, or `Referer` header, it is sufficient to authenticate any subsequent request for the lifetime of the iSpy process.

F-04 – No rate limiting on authentication failures (High). *CWE-307: Improper Restriction of Excessive Authentication Attempts.* Neither the desktop `CheckPassword` flow nor the HTTP `CheckAuth` method tracks failed attempts, applies progressive delay, or imposes a lockout. The codebase contains no symbols related to throttling. An online attacker can submit attempts as fast as the listener will accept connections.

F-05 – Outbound MD5 Digest authentication for IP cameras (Informational).

`Utilities/ConnectionFactory.cs` implements HTTP Digest authentication for outbound camera requests using MD5. This is a property of the HTTP Digest specification rather than a defect of iSpy; it is recorded here for completeness and is out of scope for remediation.

C. Dynamic verification

Dynamic confirmation of each finding is summarised in Table II.

TABLE II
DYNAMIC VERIFICATION OF STATIC FINDINGS

Finding	Verification action	Result
F-01	Inspect generated config file after creating an admin account; run <code>EncDec.DecryptData</code> on the stored value with the in-file <code>EncryptCode</code> .	Plaintext password recovered.
F-02	Search source for the literal "Salt" and the GUID <code>582df...a888</code> .	Three call sites confirmed.
F-03	Trigger video clip playback while observing outbound URLs.	<code>&auth=<GUID></code> present in URL.
F-04	<code>grep</code> for lockout / throttling identifiers across all .cs files (excl. third-party).	No matches; no logic exists.

D. Implemented fix 1 – PBKDF2 password hashing

The `EncDec`-based credential store was replaced with a new `PasswordHasher` class using PBKDF2-HMAC-SHA256 with a 128-bit per-record CSPRNG salt and 100,000 iterations (current OWASP guidance for PBKDF2-SHA256). The relevant new logic is shown in Listing 6.

```
var salt = new byte[SaltSize];  
using (var rng = RandomNumberGenerator.Create())  
    rng.GetBytes(salt);  
using (var pbkdf2 = new Rfc2898DeriveBytes(  
    password, salt, Iterations, HashAlgorithmName.SHA256)  
    )  
{  
    hash = pbkdf2.GetBytes(HashSize);  
}  
return Prefix + Iterations + "$"  
    + Convert.ToBase64String(salt) + "$"  
    + Convert.ToBase64String(hash);
```

Listing 6. Core of the new `PasswordHasher.cs`

`CheckPassword.DoCheckPassword` was modified to call `PasswordHasher.Verify`, with a fallback path that detects legacy AES-encrypted records (no PBKDF2\$ prefix) and transparently re-hashes them on the next successful login, avoiding a forced password reset for existing installations.

A reference implementation of the same algorithm in Python was used to validate the format and behaviour of the new hasher offline. Per-call verification cost was measured at approximately 101 ms on a commodity Windows host, which caps an attacker's online guess rate at roughly 9 per second per core – a reduction of several orders of magnitude relative to the original instant-equality check, before considering the F-04 throttle.

E. Implemented fix 2 – IP-based login throttle

A new `Server/LoginThrottle.cs` class was added, backed by a `ConcurrentDictionary` keyed on remote IP. Five failed `CheckAuth` comparisons within a 10-minute sliding window trigger a 15-minute HTTP 429 lockout for that IP; a successful auth clears the counter. The integration changes to `LocalServer.cs` are summarised in Listing 7.

```

1 private static bool CheckAuth(
2     string sPhysicalFilePath, string remoteIp)
3 {
4     if (LoginThrottle.IsLockedOut(remoteIp))
5         return false;
6     bool ok = GetVar(sPhysicalFilePath, "auth")
7         == MainForm.Identifier;
8     if (ok) LoginThrottle.RecordSuccess(remoteIp);
9     else LoginThrottle.RecordFailure(remoteIp);
10    return ok;
11 }

```

Listing 7. Modified `CheckAuth` integrating the throttle

The 429 response body uses an `errorType` of `ratelimit` to allow the front-end JavaScript to distinguish lockout from a normal auth failure without exposing additional information about the account.

F. Issues left unfixed and why

F-02 is partly mitigated by Fix 1 for the user-account case, because the `EncDec` salt is no longer used for that data path. The hardcoded GUID protecting `WSPassword` remains in place; a proper fix would migrate that field to DPAPI (`ProtectedData.Protect`), which scopes the encryption key to the current Windows user account and removes the hardcoded literal entirely. This change touches the upstream web service handshake and was deemed out of scope for this assessment. F-03 (token in URL) is similarly not patched because the change requires migrating the embed URLs in `CameraWindow.cs` and `VolumeLevel.cs` to use the `Authorization` header or a short-lived signed-URL scheme; either is a non-trivial protocol change.

VI. SUMMARY AND LESSONS LEARNED

A. Summary of findings

TABLE III
CONSOLIDATED FINDINGS

ID	Vulnerability	CWE	Severity	Fixed
F-01	Reversible password storage	AES 257	Critical	Yes
F-02	Hardcoded salt and master key	760, 798	Critical	Partial
F-03	Auth token in URL query string	598	Medium	No
F-04	No login rate limiting / lockout	307	High	Yes
F-05	MD5 in outbound HTTP Digest	–	Info	No

B. Lessons learned

Encryption is not authentication. The most consequential defect found was the use of a reversible cipher, `EncDec`, to store something that should never have been recoverable in the first place. Once a password can be decrypted, every other control around it (file permissions, the per-installation `EncryptCode`, the operating system’s user account model) is reduced to “trust the file system,” and the moment that assumption fails the credential is gone. The lesson is to use a one-way password-hashing primitive unconditionally; if a system needs to recover the original password, the design itself is wrong.

A salt that ships in the binary is not a salt. The literal “Salt” in `EncDec.cs` is a useful illustration of how cryptographic guidance fails when implemented mechanically. The author appears to have known a salt was required and produced something that satisfied the API, but the value is global, four bytes, and visible to every attacker. The same lesson applies to the hardcoded GUID guarding `WSPassword`: secrecy of source and secrecy of keys are different things.

Throttling is not optional. The complete absence of failed-attempt accounting is unusual for a network-facing service in 2024 and is a direct consequence of writing the HTTP listener by hand instead of building on a framework that provides it for free. Any custom authentication path needs a corresponding throttle, and the throttle needs to be the first thing the auth function does, not the last.

Threat modelling pays for itself. Drafting the STRIDE table in Section III before reading any code directed our attention to the information-disclosure and elevation-of-privilege rows, which is where all four of the meaningful findings landed. Without that step, the assessment would likely have spent its time on the larger but less-impactful camera and recording paths.

SUPPLEMENTARY MATERIALS

The following files accompany this submission and contain the implementation artefacts referenced in Section V:

- `Fix1_PasswordHashing.txt` – the new `PasswordHasher.cs` source and unified diffs against `CheckPassword.cs`, `Controls/PermissionsForm.cs`, and `MainForm_Configuration.cs`.
- `Fix2_LoginThrottle.txt` – the new `Server/LoginThrottle.cs` source and unified diffs against `Server/LocalServer.cs`.
- `test_password_hasher.py` – a reference implementation of the PBKDF2 hasher used to validate the format and per-call cost reported in Section V-C.

REFERENCES

- [1] iSpy Open Source Project, GitHub Repository. [Online]. Available: <https://github.com/ispysoftware/iSpy>
- [2] iSpy Official Website. [Online]. Available: <https://www.ispyconnect.com/>
- [3] MITRE, “CWE-257: Storing Passwords in a Recoverable Format.” [Online]. Available: <https://cwe.mitre.org/data/definitions/257.html>

- [4] MITRE, “CWE-307: Improper Restriction of Excessive Authentication Attempts.” [Online]. Available: <https://cwe.mitre.org/data/definitions/307.html>
- [5] MITRE, “CWE-598: Information Exposure Through Query Strings in GET Request.” [Online]. Available: <https://cwe.mitre.org/data/definitions/598.html>
- [6] MITRE, “CWE-760: Use of a One-Way Hash with a Predictable Salt.” [Online]. Available: <https://cwe.mitre.org/data/definitions/760.html>
- [7] MITRE, “CWE-798: Use of Hard-coded Credentials.” [Online]. Available: <https://cwe.mitre.org/data/definitions/798.html>
- [8] OWASP, “Password Storage Cheat Sheet.” [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
- [9] OWASP, “Authentication Cheat Sheet.” [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html
- [10] OWASP, “Top Ten Web Application Security Risks,” 2021. [Online]. Available: <https://owasp.org/www-project-top-ten/>
- [11] NIST, “Digital Identity Guidelines,” SP 800-63B, 2017.
- [12] NIST, “Recommendation for Password-Based Key Derivation,” SP 800-132, 2010.
- [13] A. Shostack, *Threat Modeling: Designing for Security*. Wiley, 2014.
- [14] M. Howard and D. LeBlanc, *Writing Secure Code*, 2nd ed. Microsoft Press, 2002.